LANGUES DE SPÉCIALITÉ

Utiliser GitHub pour le projet de cours

Guide pas à pas pour débuter — navigateur, GitHub Desktop et fichiers du projet

Enseignant : Francis Lareau

Dépôt de départ : FrancisLa/Langues_de_specialite

Le parcours retenu
Consulter le dépôt du cours → créer un dépôt privé → travailler sur une copie locale → récupérer les nouveautés → modifier et enregistrer → créer une version → l'envoyer sur GitHub.

Ce guide présente uniquement les opérations utiles au projet du cours. Il propose GitHub Desktop pour éviter d'utiliser des commandes dans le terminal. Il ne nécessite pas de connaissances préalables en Git. Les intitulés anglais des boutons sont conservés pour vous aider à les retrouver à l'écran ; leur emplacement peut varier légèrement selon la version.

Le guide a été adapté au dépôt public « FrancisLa/Langues_de_specialite ». Dans sa structure actuelle, les dossiers « journal/ », « chaine/ », « donnees/ », « article/ », etc. se trouvent directement à la racine. Certains noms de fichiers sont proposés dans les README à titre d'exemples : si un fichier n'existe pas encore, vous le créerez au moment opportun.

1. Comprendre et repérer

1.1 Ce que GitHub fera dans ce cours

GitHub héberge les fichiers partageables de votre enquête ainsi que leur historique. Vous l'utiliserez pour conserver la documentation de la chaîne de recherche, suivre les changements, collaborer et permettre à l'enseignant de consulter le projet. Il ne remplace ni votre environnement Python ni votre espace de stockage autorisé pour les corpus.

Outil ou document	Rôle dans le cours
GitHub dans le navigateur	Lire les consignes ; créer le dépôt personnel (ou d'équipe); inviter les collaborateurs ; vérifier les fichiers envoyés.
GitHub Desktop	Récupérer une copie locale ; examiner les changements ; créer et envoyer des versions ; recevoir le travail de l'équipe.
Colab, Spyder, Jupyter, Word ou votre éditeur	Modifier les scripts, les notebooks et les documents sur votre ordinateur. GitHub Desktop ne les exécute pas.
Journal de recherche	Expliquer pourquoi les décisions évoluent, y compris les usages substantiels de l'IA et les vérifications qui y sont associées.
Article IMRAD	Présenter la réponse scientifique ; il reste un fichier de votre projet, pas un formulaire GitHub.

1.2 Six mots indispensables

Mot affiché	À comprendre
Repository / dépôt	Le dossier du projet avec ses fichiers et son historique.
Clone / cloner	Créer une copie de travail liée à votre dépôt en ligne. Ce n'est pas un simple téléchargement ZIP.
Commit / version enregistrée	Un ensemble de changements, accompagné d'un message décrivant ce qui a été fait.
Push origin / envoyer	Transférer sur GitHub les commits présents sur votre ordinateur.
Fetch origin / vérifier les nouveautés	Demander à GitHub si de nouvelles versions sont disponibles.
Pull origin / récupérer	Intégrer sur votre ordinateur les versions disponibles sur GitHub.

Enregistrer, commiter, envoyer : trois actions différentes
Ctrl+S enregistre le fichier sur votre ordinateur. Commit to main enregistre une version locale. Push origin transmet cette version au dépôt en ligne. Pour que l'étudiant et l'enseignant puissent voir votre travail, les trois actions doivent avoir été effectuées.

Dans ce parcours, vous travaillez tous sur la branche commune nommée « main ». Ce nom apparaît dans GitHub Desktop. Il n'est pas nécessaire de créer ou de gérer d'autres branches pour suivre ce guide.

1.3 Se repérer dans le dépôt du cours

1. Ouvrez le lien du dépôt de départ dans votre navigateur.

2. Lisez le README affiché sous la liste des fichiers. Il présente le projet et ses livrables.

3. Cliquez sur le dossier correspondant à votre besoin, puis sur son README.md.

4. Utilisez le chemin en haut de la liste pour revenir au dossier parent ou à la racine du dépôt.

Dossier actuel	Ce que vous y cherchez ou y rangerez
docs/	Documents communs, guides et ressources du cours.
journal/	Journal de bord et bilans selon les consignes.
chaine/	Question, corpus, acquisition, préparation, analyse, validation et interprétation.
donnees/	Inventaire, exclusions, dictionnaire des métadonnées et règles d'accès au corpus.
environnement/	Logiciels, dépendances et versions nécessaires.
configuration/	Paramètres et modèles de configuration partageables.
resultats/	Tableaux, figures et extraits partageables.
article/	Article IMRAD, bibliographie et annexes.

Votre travail sera effectué dans votre dépôt personnel (ou d'équipe), pas dans « FrancisLa/Langues_de_specialite ». Les mises à jour du dépôt du professeur ne seront pas automatiquement répliquées dans votre dépôt : lisez les annonces du cours et appliquez les changements demandés à vos propres fichiers.

2. Créer le dépôt personnel (ou d'équipe)

2.1 Préparer les outils

1. Chaque étudiant crée son propre compte GitHub s'il n'en a pas. Notez votre nom d'utilisateur, sans partager votre mot de passe.

2. Installez GitHub Desktop depuis desktop.github.com et ouvrez l'application.

3. Choisissez « Sign in to GitHub.com » et terminez la connexion dans le navigateur.

4. Si GitHub Desktop demande un nom et une adresse pour attribuer les commits, utilisez les informations de votre compte et respectez vos paramètres de confidentialité.

5. Si vous travaillez en équipe, désignez une personne pour créer le dépôt. Les autres le rejoindront ensuite avec leur propre compte.

Ne créez qu'un seul dépôt commun par équipe. Il sera privé. Le propriétaire du dépôt peut ajouter les autres membres s'il y en a, ainsi que **FrancisLa** comme collaborateurs ; sur un dépôt personnel, ces accès permettent de contribuer. L'enseignant est invité à consulter et à évaluer, sans que vous ayez à rendre les contenus publics.

2.2 Télécharger les fichiers de départ

1. Sur le dépôt du professeur « https://github.com/FrancisLa/Langues_de_specialite », cliquez sur le bouton vert « Code ».

2. Choisissez « Download ZIP » et enregistrez l'archive dans un emplacement temporaire.

3. Décompressez l'archive avec « Extraire tout… » sous Windows, ou avec l'outil habituel de votre système.

4. Ouvrez le dossier décompressé. Vérifiez qu'il contient README.md, .gitignore, LICENSE, LICENSE-DOCS.md et les dossiers principaux.

Le ZIP n'est qu'une copie des fichiers au moment du téléchargement ; il ne relie pas votre travail au dépôt en ligne et ne contient pas l'historique complet de celui-ci. Vous allez utiliser ces fichiers pour initialiser un nouveau dépôt personnel.

2.3 Créer votre dépôt privé dans le navigateur

1. Connectez-vous à votre compte GitHub dans votre navigateur. Utilisez le bouton +, puis « New repository ».

2. Dans « Owner », choisissez votre compte (ou le compte du membre responsable de la création pour les équipes).

3. Dans « Repository name », utilisez un nom court, par exemple LSP_nom_prenom. Le nom est un exemple, pas une consigne de numérotation imposée.

4. Dans « Description », écrivez une phrase sur le projet, même provisoire.

5. Choisissez « Choose visibility » → « Private ». Activez « Add README » pour initialiser le dépôt.

6. Ne choisissez pas de .gitignore ni de licence supplémentaires à cette étape : les fichiers du gabarit seront copiés ensuite.

7. Cliquez sur « Create repository ». Vérifiez que vous êtes dans VOTRECOMPTE/LSP_nom_prenom et que la mention « Private » apparaît.

2.4 Cloner votre dépôt avec GitHub Desktop

1. Dans GitHub Desktop : « File » → « Clone repository » → « URL ».

2. Copiez l'adresse de votre nouveau dépôt personnel (ou d'équipe) dans le champ URL. Ne collez pas celle du professeur.

3. Dans « Local Path », choisissez un emplacement sur votre ordinateur que vous pourrez facilement retrouver. Si Google Drive est installé, vous pouvez choisir, par exemple, « **G:/My Drive/LSP_nom_prenom** » ou « **G:/Mon disque/LSP_nom_prenom** ». Son installation est facultative, mais nous verrons durant le cours en quoi elle peut être utile.

4. Cliquez sur « Clone ». Attendez la fin de l'opération.

5. Dans Desktop, vérifiez le nom de « Current Repository ». Puis choisissez « Repository » → « Show in Explorer » pour ouvrir le dossier local ; sur macOS, utilisez « Show in Finder ».

> Le dossier local est votre dossier de travail
> Ouvrez vos documents, scripts et notebooks depuis ce dossier, pas depuis le ZIP ou un dossier Téléchargements.
> C'est le dossier qui contient directement chaine/, donnees/, configuration/, environnement/, etc.

2.5 Copier le gabarit dans le dépôt local

1. Ouvrez côte à côte le dossier décompressé du cours et le dossier local de votre projet.

2. Copiez le CONTENU du dossier du cours dans votre dossier (ou celui de l'équipe) : tous les dossiers, README.md, .gitignore, LICENSE et LICENSE-DOCS.md.

3. Acceptez le remplacement du README initial de votre nouveau dépôt par celui du gabarit.

4. Ne copiez pas le dossier extérieur Langues_de_specialite-main comme sous-dossier : les dossiers « chaine/ », « journal/ », etc. doivent être directement à la racine.

5. Conservez le dossier technique .git créé par le clonage ; ne le supprimez pas et ne le remplacez pas.

6. Assurez-vous que .gitignore a bien été copié. Sur macOS, les fichiers commençant par un point peuvent être masqués : Cmd+Maj+. permet de les afficher.

```
LSP_Nom_Prenom/
├── README.md
├── .gitignore
├── LICENSE
├── LICENSE-DOCS.md
├── article/
├── chaine/
├── configuration/
├── docs/
├── donnees/
├── environnement/
├── journal/
└── resultats/
```

Les fichiers de licence et les mentions de Francis Lareau accompagnent les contenus repris du gabarit. Ne les remplacez pas simplement par votre nom. Vous préciserez la licence de vos propres productions selon les consignes, tout en conservant les avis applicables aux contenus réutilisés.

7. Revenez dans Desktop, onglet « Changes ». Examinez la liste des fichiers ajoutés.

8. Dans « Summary », écrivez : Initialise le projet à partir du gabarit du cours.

10. Cliquez sur « Commit … to main », puis sur « Push origin ».

11. Dans le navigateur, actualisez votre dépôt : les dossiers doivent maintenant apparaître.

2.6 Inviter l'enseignant (et l'équipe le cas échéant)

1. Sur la page du dépôt personnel (ou d'équipe) : « Settings » → « Collaborators », dans la rubrique « Manage access ».

2. Cliquez sur « Add people ». Ajoutez « FrancisLa » pour donner accès à l'enseignant.

3. Les étapes suivantes s'appliquent aux équipes. Recherchez le nom d'utilisateur exact de chaque équipier, puis envoyez l'invitation.

4. Chaque personne doit accepter l'invitation reçue ; une invitation en attente n'accorde pas encore d'accès.

5. Chaque équipier clone ensuite le dépôt de l'équipe avec GitHub Desktop. Il ne refait pas les étapes de création et de copie du fichier ZIP.

Chaque membre dispose désormais de sa propre copie locale, reliée au même dépôt privé. Notez l'URL commune et le chemin local de chacun. Ne partagez pas un seul compte GitHub.

3. Travailler au quotidien

3.1 Le cycle à suivre à chaque séance

Moment	Action	Vérification
Avant de modifier	Ouvrir le bon dépôt dans Desktop ; cliquer Fetch origin, puis Pull origin si le bouton est proposé.	Votre copie contient les dernières modifications de l'équipe.
Pendant le travail	Ouvrir les fichiers depuis le dossier cloné ; modifier dans l'éditeur approprié ; enregistrer.	Le fichier a le bon nom et se trouve au bon endroit.
Avant le commit	Examiner Changes ; lire les différences quand elles sont affichables ; sélectionner les fichiers utiles.	Aucun secret, corpus interdit ou fichier accidentel n'est sélectionné.
Créer une version	Écrire un résumé précis ; cliquer Commit to main.	La version apparaît dans History sur votre ordinateur.
Partager	Cliquer Push origin ; vérifier sur GitHub dans le navigateur.	Le travail est visible par les personnes autorisées.

« Fetch origin » vérifie les nouveautés ; s'il y a des changements distants, « Pull origin » les récupère. « Push origin » envoie vos commits locaux. Si Desktop vous demande de traiter des modifications non enregistrées avant la récupération, arrêtez-vous et suivez les consignes de dépannage : ne supprimez rien pour faire disparaître le message.

3.2 Le minimum de Markdown utile au cours

Un fichier **.md** est un fichier texte. GitHub en affiche une version mise en forme. Pour les README, protocoles et journaux, les conventions suivantes suffisent pour commencer.

```
# Titre principal

## Sous-titre

Un paragraphe de texte.

- Premier élément
- Deuxième élément
```

```
[Inventaire](../donnees/inventaire.csv)
```

Une ligne vide sépare les paragraphes. Les **#** servent aux titres et le tiret aux listes. Dans l'exemple de lien, **../** signifie « remonter d'un dossier » : il convient depuis journal/, mais pas forcément depuis un sous-dossier profond de chaine/. Copiez un lien vers un fichier sur GitHub si vous n'êtes pas encore à l'aise avec les chemins relatifs. Un fichier texte ne doit pas être enregistré comme un document Word : utilisez Word pour les fichiers .docx de l'article. **VS Code** est un éditeur permettant d'éditer les fichiers Markdown. Il est un peu comme un traitement de texte spécialisé pour la programmation. Au lieu d'écrire un document, on y écrit du code Python, R, HTML, etc., et on peut ensuite l'exécuter directement depuis l'environnement.

3.3 Premier exemple : tenir le journal

1. Récupérez les changements (**Fetch origin** dans **Desktop**).

2. Ouvrez le dossier local de journal/. Lisez son README.

3. Créez **journal/journal_de_bord.md** s'il n'existe pas. Créez-le comme fichier texte avec l'extension « **.md** », et non pas « .md**.txt** ». Ouvrez le dans votre éditeur de texte (nous recommandons **VS Code** comme éditeur).

4. Ajoutez une nouvelle entrée avec une date, un objectif et identifiez les personnes concernées lorsque le travail est collectif). Notez ce qui a été fait, les questionnements, les observations, les solutions, les décisions, les justifications, etc. La consigne du dépôt prévoit au moins une entrée hebdomadaire et les décisions importantes ; les usages substantiels d'IA doivent y être documentés avec leurs vérifications. Enfin, enregistrez.

5. Dans **Desktop**, vérifiez le chemin du fichier et lisez les modifications affichées.

6. Écrivez dans **Summary** : Précise la question sur les fonctions des citations. Cliquez **Commit to main**, puis **Push origin**.

7. Actualisez le dépôt dans le navigateur et ouvrez ce même fichier pour confirmer l'envoi.

```
## 2026-10-27 — Délimitation du corpus

Personnes concernées : [noms]

Observation : notre objet initial couvre trop de genres.
Décision : limiter le pilote aux rapports d'expertise.
Justification : cette restriction rend la comparaison réalisable.
Vérification prévue : examiner cinq rapports accessibles.
Trace : chaine/02_conception_du_corpus/plan_corpus.md
```

Il s'agit d'un exemple construit. Remplacez son contenu par le vôtre. Commit possible : Documente la réduction du corpus pilote. Le commit décrit les fichiers changés ; le journal explique la décision scientifique. L'un ne remplace pas l'autre.

3.4 Choisir un message de commit utile

À éviter	À préférer
modifs	Précise les critères d'inclusion des rapports
final	Ajoute la version PDF de l'article pour relecture
test	Corrige la lecture UTF-8 du fichier de métadonnées
travail du jour	Documente la vérification de dix annotations

Un commit peut regrouper plusieurs fichiers liés à une même tâche. Ne créez pas artificiellement des commits pour augmenter leur nombre. Il est possible de conserver une étape provisoire si le message et le journal indiquent honnêtement son état ; n'annoncez pas une analyse réussie si elle n'a pas été vérifiée.

3.5 Deuxième exemple : préciser la question

1. Récupérez les changements (**Fetch origin** dans **Desktop**).

2. Ouvrez le dossier local de chaine/01_question_et_cadre/. Lisez son README.

3. Ouvrez **question_recherche.md** dans votre éditeur de texte. S'il n'existe pas, créez-le comme fichier texte avec l'extension **.md**.

4. Ajoutez un objet provisoire, une question et une incertitude à vérifier. Enregistrez.

5. Dans **Desktop**, vérifiez le chemin du fichier et lisez les modifications affichées.

6. Écrivez dans **Summary** : Précise la question sur les fonctions des citations. Cliquez **Commit to main**, puis **Push origin**.

7. Actualisez le dépôt dans le navigateur et ouvrez ce même fichier pour confirmer l'envoi.

```
# Question de recherche

## Objet provisoire
Les fonctions des citations dans les rapports d'expertise.

## Question provisoire
Comment les citations soutiennent-elles les recommandations dans les rapports retenus ?

## À vérifier
Quelles sources sont accessibles et quels rapports relèvent du périmètre ?
```

Il s'agit d'un exemple construit. Remplacez son contenu par votre propre question ; ne le présentez pas comme votre recherche déjà effectuée.

3.6 Se coordonner sans compliquer le travail

- Avant de travailler, dites à l'équipe quels fichiers vous allez modifier ; évitez de modifier le même fichier en même temps.
- Récupérez les nouveautés au début et envoyez vos versions à la fin de chaque période de travail.
- Pour journal_de_bord.md et l'article Word, convenez d'un tour de rédaction ; les fichiers .docx et .ipynb peuvent être difficiles à réconcilier automatiquement.
- Les autres membres utilisent **Pull origin** pour recevoir le travail modifié. Les échanges de pièces jointes par messagerie ne remplacent pas cette synchronisation.
- Ne mettez pas le dossier cloné dans un dossier synchronisé à plusieurs par un autre service pour remplacer Git. Chaque membre garde sa propre copie de travail.

En cas de conflit ou d'envoi refusé
N'effacez aucun fichier et n'utilisez pas Discard Changes, une suppression du dépôt ou une réécriture de l'historique pour « débloquer ». Conservez vos fichiers, notez le message exact et demandez une aide à l'enseignant. Reprenez le travail après clarification ; vous n'avez pas à apprendre les procédures avancées de résolution pour suivre ce guide.

4. Ranger et protéger les fichiers

4.1 Choisir le bon emplacement

Votre fichier	Emplacement à utiliser
Question et définitions	chaine/01_question_et_cadre/
Plan du corpus et critères	chaine/02_conception_du_corpus/
Script ou protocole de collecte	chaine/03_acquisition/
Extraction, nettoyage, segmentation, annotation ou représentation	Sous-dossier correspondant de chaine/04_preparation/
Analyse et contrôles	chaine/05_analyse/ et chaine/06_validation/
Interprétation, contre-exemples et limites	chaine/07_interpretation/
Registre des documents et métadonnées	donnees/inventaire.csv et donnees/dictionnaire_metadonnees.md
Tableaux, figures et extraits finaux autorisés	resultats/
Article Word et PDF	article/article_imrad.docx et article/article_imrad.pdf
Décisions de recherche et usages d'IA	journal/journal_de_bord.md

N'ajoutez pas automatiquement tous les fichiers cités comme exemples dans les README. Créez les documents nécessaires à votre enquête et documentez les opérations effectivement réalisées. Un dossier peut ne pas être visible sur GitHub tant qu'il ne contient aucun fichier suivi : créez un README utile si sa documentation est nécessaire, évitez les dossiers vides.

4.2 Comprendre le .gitignore du cours

Le fichier .gitignore permet d'exclure du transfert Desktop -> GitHub certains dossiers ou fichiers. Celui fourni dans le dépôt exclut notamment donnees/sources/, donnees/intermediaires/, donnees/preparees/, les environnements locaux comme .venv/, les fichiers .env, certaines configurations locales et les fichiers .log. Un fichier ignoré peut exister sur votre ordinateur sans apparaître dans Changes ni sur GitHub. C'est souvent volontaire, pas une panne.

Conserver dans le dépôt, si autorisé	Garder hors du dépôt selon les règles du projet
README, protocoles et journal	Mots de passe, clés API, jetons et fichiers .env
Scripts .py et notebooks .ipynb vérifiés	Environnement Python installé, fichiers temporaires et caches
Inventaire et métadonnées partageables	Textes complets, PDF ou autres données non redistribuables
Versions des dépendances et paramètres partageables	Sorties contenant des données personnelles ou confidentielles
Résultats finaux et article autorisés	Données volumineuses ou intermédiaires prévues pour un stockage séparé

Ignorer n'est pas sécuriser

.gitignore n'efface pas un fichier déjà suivi et ne remplace pas une vérification humaine. Ne versez pas un fichier exclu directement par le navigateur pour forcer sa présence. Si une clé ou un contenu confidentiel a été envoyé, arrêtez sa diffusion et contactez l'enseignant ; une suppression visible ne garantit pas son retrait de l'historique. Une clé exposée doit être révoquée auprès du service concerné.

Même un inventaire ou une sortie de notebook peut contenir des informations sensibles. Inspectez les contenus avant le commit. Les données non publiées restent dans l'espace de stockage autorisé du projet ; indiquez leur provenance et les conditions d'accès dans donnees/README.md. GitHub n'est pas une sauvegarde des fichiers ignorés : leur conservation doit être organisée séparément.

4.3 Importer un script, un notebook ou un article

1. Copiez ou enregistrez le fichier directement dans le dossier local approprié de votre dépôt.

2. Pour un notebook téléchargé du cours, vérifiez qu'il porte bien l'extension .ipynb ; ouvrez-le dans VS Code pour l'exécuter, ou via Colab sur GitHub.

3. Après le travail, enregistrez le fichier. Vérifiez les sorties, notamment l'absence de clés, données personnelles ou contenus non partageables.

4. Revenez dans Desktop. Sélectionnez seulement les fichiers utiles et autorisés ; créez un commit explicite puis envoyez-le.

5. Dans le navigateur, vérifiez le nom, le chemin et la présence du fichier. Un rendu imparfait du notebook ne prouve pas une erreur du code ; le test se fait dans l'environnement du projet.

Le PDF et le Word de l'article sont tous deux utiles dans ce cours. Ils peuvent être suivis dans Git, même si Desktop ne montre pas leurs différences ligne par ligne. En équipe, choisissez qui les modifie à un moment donné pour éviter des versions concurrentes.

5. Vérifier, remettre et dépanner

5.1 Vérifier ce qui a été envoyé

1. Dans Desktop, regardez l'onglet History pour retrouver vos derniers commits et leurs messages.

2. Cliquez sur un commit : vérifiez les fichiers concernés et les différences lorsque le format permet leur affichage.

3. Assurez-vous d'avoir utilisé Push origin après le commit.

4. Ouvrez le dépôt personnel (ou d'équipe) sur GitHub, actualisez la page et contrôlez le fichier concerné.

Un commit possède un identifiant unique appelé hash ou SHA. Cet identifiant sert à désigner précisément l'état évalué du projet. Un message de commit n'est pas une preuve que le programme fonctionne : les vérifications scientifiques restent à documenter dans la chaîne de recherche.

5.2 Remettre le projet

1. Enregistrez tous les fichiers utiles, puis vérifiez le journal, la chaîne, les résultats et le PDF final de l'article.

2. Dans Desktop, examinez Changes et assurez-vous qu'aucune modification destinée à la remise n'a été oubliée.

3. Créez un commit, par exemple Prépare la remise finale du projet, puis Push origin.

4. Sur GitHub, vérifiez que FrancisLa a accepté l'invitation et peut accéder au dépôt privé. N'affichez pas le projet publiquement pour résoudre un simple problème d'accès.

5. Depuis la page du dépôt, ouvrez la liste des commits près du dernier message. Ouvrez le commit final et copiez son lien ; utilisez le bouton de copie du SHA s'il est proposé.

6. Lors de la remise, envoyez un courriel au professeur indiquant votre nom complet, l'URL de votre dépôt, le lien ou SHA du commit final et le chemin article/article_imrad.pdf.

Le commit identifie la version remise ; les dates et le mode exact de dépôt suivent le syllabus. Ne créez pas une nouvelle version après la remise en supposant qu'elle remplace automatiquement la version évaluée. Si une correction est demandée, transmettez un nouvel identifiant et expliquez le changement.

5.3 Problèmes fréquents : que vérifier en premier ?

Problème	Vérification et action sûre
Je ne vois pas mes modifications dans Desktop.	Avez-vous enregistré le fichier ? Travaillez-vous dans le dossier cloné et le bon Current Repository ? Le fichier est-il ignoré ?
J'ai fait un commit, mais l'équipe ne voit rien.	Cliquez Push origin, puis vérifiez le dépôt en ligne. Les équipiers devront récupérer les changements.
Je ne vois pas le travail d'un équipier.	Vérifiez qu'il a envoyé ses commits ; faites Fetch origin puis Pull origin.
Un dossier de données ne s'affiche pas.	Consultez .gitignore. Les trois dossiers de données sources, intermédiaires et préparées sont exclus dans le gabarit actuel. Ne forcez pas leur publication.
Je n'ai pas accès au dépôt privé.	Vérifiez le compte connecté, l'invitation et son acceptation. Demandez au propriétaire de vérifier les collaborateurs.
Desktop refuse de récupérer ou d'envoyer.	Ne supprimez rien. Enregistrez les fichiers, relevez le message exact et demandez de l'aide avant de poursuivre.
J'ai créé README.md.txt au lieu de README.md.	Dans l'explorateur, affichez les extensions des fichiers puis corrigez le nom ; vérifiez le changement dans Desktop avant envoi.
Le notebook ne s'exécute pas sur GitHub.	C'est normal : ouvrez la copie locale dans VS Code ou Colab et utilisez l'environnement documenté du projet.
J'ai envoyé un secret ou une donnée interdite.	Ne comptez pas sur une suppression ou sur .gitignore. Contactez immédiatement l'enseignant ; révoquez la clé exposée s'il s'agit d'un accès.

5.4 Vérification avant et après chaque envoi

- Je suis dans mon dépôt personnel (ou d'équipe), pas dans celui du professeur.
- J'ai enregistré mes fichiers dans le dossier local cloné.
- Je comprends les fichiers sélectionnés dans Changes ; aucun secret ni contenu interdit n'est inclus.
- Le message décrit honnêtement la modification.
- J'ai fait Commit to main puis Push origin.
- Je peux retrouver le fichier en ligne et expliquer le changement dans le journal si nécessaire.

5.5 Mémo de travail

```
DÉBUT
Desktop → bon dépôt → Fetch origin → Pull origin si proposé

TRAVAIL
Ouvrir les fichiers locaux → modifier → enregistrer

PARTAGE
Changes → vérifier → message → Commit to main → Push origin

CONTRÔLE
Navigateur → bon dépôt → vérifier les fichiers
```

```
REMETTRE
URL du dépôt + commit final + article/article_imrad.pdf
```

Repères documentaires utilisés

Le guide s'appuie sur les fichiers publics du cours et sur la documentation officielle de GitHub consultés le 27 octobre 2026. Le parcours ZIP → dépôt privé → GitHub Desktop est une proposition pédagogique pour débutants ; il ne suppose pas qu'un bouton de création à partir d'un modèle soit activé dans le dépôt du professeur.

Dépôt et README général du cours

README du journal : décisions et usages d'assistance

README de l'article : livrable et version finale

Règles .gitignore effectivement fournies

GitHub : télécharger une archive ZIP

GitHub Desktop : cloner un dépôt

GitHub Desktop : examiner, commiter et envoyer

GitHub : inviter des collaborateurs